AI Assisted Coding
Lab Assignment - 13

Name : Shivanand Rama
Hall.t.No. : 2303A51037
Batch No. : 01

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions
Lab Objectives:

- Identify code smells and inefficiencies in legacy Python scripts.
- Use AI-assisted coding tools to refactor for readability, maintainability, and performance.
- Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Code Duplication)

- **Task:** Use AI to refactor a given Python script that contains multiple repeated code blocks.
- **Instructions:**
  - Prompt AI to identify duplicate logic and replace it with functions or classes.
  - Ensure the refactored code maintains the same output.
  - Add docstrings to all functions.
- **Sample Legacy Code:**

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
```

print("Perimeter of Rectangle:", 2 * (10 + 15))

- **Expected Output:**
  - o Refactored code with a reusable function and no duplication.
  - o Well documented code

Copilot suggested code :

```python
task_1.py > ...
# Use AI to refactor a given Python script that contains multiple repeated code blocks.
# Legacy script with repeated logic
# print("Area of Rectangle:", 5 * 10)
# print("Perimeter of Rectangle:", 2 * (5 + 10))
# print("Area of Rectangle:", 7 * 12)
# print("Perimeter of Rectangle:", 2 * (7 + 12))
# print("Area of Rectangle:", 10 * 15)
# print("Perimeter of Rectangle:", 2 * (10 + 15))

# Refactored script with reusable function and no duplication
def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
# List of rectangles with their dimensions
rectangles = [(5, 10), (7, 12), (10, 15)]
# Calculate and print the properties of each rectangle
for length, width in rectangles:
    area, perimeter = calculate_rectangle_properties(length, width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_1.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\AI Assist\lab_assignment-13>
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- **Task:** Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- **Instructions:**
  - Identify repeated or related logic and extract it into reusable functions.
  - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- **Sample Legacy Code:**

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

- **Expected Output:**
  - Code with a function calculate_total(price) that can be reused for multiple price inputs.
  - Well documented code

Copilot suggested code :

```
task_2.py > ...
# Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
# Legacy script with inline repeated logic
# price = 250
# tax = price * 0.18
# total = price + tax
# print("Total Price:", total)
# price = 500
# tax = price * 0.18
# total = price + tax
# print("Total Price:", total)

# Refactored script with reusable function and no duplication
def calculate_total(price):
    """
    Calculate the total price including tax.

    Parameters:
    price (float): The original price of the item.

    Returns:
    float: The total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# List of prices to calculate total for
prices = [250, 500]
# Calculate and print the total price for each price input
for price in prices:
    total_price = calculate_total(price)
    print(f"Total Price: {total_price}")
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_2.py"
Total Price: 295.0
Total Price: 590.0
PS C:\AI Assist\lab_assignment-13>
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic) Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
    - o Accept marks as a parameter.
    - o Return the corresponding grade as a string.
    - o The grading logic must strictly follow the conditions below:
        - ▪ Marks ≥ 90 and ≤ 100 → "Grade A"
        - ▪ Marks ≥ 80 → "Grade B"
        - ▪ Marks ≥ 70 → "Grade C"
        - ▪ Marks ≥ 40 → "Grade D"
        - ▪ Marks ≥ 0 → "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:
    - o The class
    - o The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

```
marks = 72

if marks >= 90:
        print("Grade A")
elif marks >= 75:
        print("Grade B")
                else:  print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

Copilot suggested code :

task_3.py > ...

```python
# The given script contains duplicated conditional statements used to assign grades
# based on student marks. This redundancy violates clean code principles and reduces
# maintainability.
# You are required to refactor the script using a class-based design to improve
# modularity, reusability, and readability while preserving the original grading
# logic.
# • Given code:
    # marks = 85

    # if marks >= 90:
    #   print("Grade A")
    # elif marks >= 75:
    #   print("Grade B")
    # else:
    #   print("Grade C")

    # marks = 72

    # if marks >= 90:
    #   print("Grade A")
    # elif marks >= 75:
    #   print("Grade B")
    # else:
    #   print("Grade C")

class GradeCalculator:
    """
    A class to calculate grades based on student marks.

    Methods
    -------
    calculate_grade(marks)
        Accepts marks as a parameter and returns the corresponding grade as a string.
    """

    def calculate_grade(self, marks):
        """
        Calculate the grade based on the provided marks.

        Parameters
        ----------
        marks : int
            The marks obtained by the student (0 to 100).

        Returns
        -------
        str
            The grade corresponding to the marks.
        """
        if 90 <= marks <= 100:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        elif marks >= 0:
            return "Fail"
        else:
            raise ValueError("Marks should be between 0 and 100.")
# Create an object of the GradeCalculator class
grade_calculator = GradeCalculator()
# Test the calculate_grade method with different marks
marks_list = [85, 72, 95, 60, 30]
for marks in marks_list:
    grade = grade_calculator.calculate_grade(marks)
    print(f"Marks: {marks} - {grade}")
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_3.py"
Marks: 85 - Grade B
Marks: 72 - Grade C
Marks: 95 - Grade A
Marks: 60 - Grade D
Marks: 30 - Fail
PS C:\AI Assist\lab_assignment-13>
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

• **Task:** Use AI to refactor procedural input–processing logic into functions.

Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• **Sample Legacy Code:**

```
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

• **Expected Output:**

o Modular code using functions like get_input(), calculate_square(), and display_result().

Copilot suggested code :

```python
# Use AI to refactor procedural input-processing logic into functions.
# Sample Legacy Code:
    # num = int(input("Enter number: "))
    # square = num * num
    # print("Square:", square)

def get_input():
    num = int(input("Enter number: "))
    return num

def calculate_square(num):
    square = num * num
    return square

def display_result(square):
    print("Square:", square)
def main():
    num = get_input()
    square = calculate_square(num)
    display_result(square)
if __name__ == "__main__":
    main()
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_4.py"
Enter number: 17
Square: 289
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_4.py"
Enter number: 27
Square: 729
PS C:\AI Assist\lab_assignment-13>
```

Task 5 (Refactoring Procedural Code into OOP Design)

• **Task:** Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Copilot suggested code :

task_5.py > ...

```python
# Use AI to refactor procedural code into a class-based design.
# Legacy Code:
    # salary = 50000
    # tax = salary * 0.2
    # net = salary - tax
    # print(net)

class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        tax = self.calculate_tax()
        return self.salary - tax
# Example usage
employee = EmployeeSalaryCalculator(50000)
net_salary = employee.calculate_net_salary()
print(net_salary)
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_5.py"
40000.0
PS C:\AI Assist\lab_assignment-13>
```

Task 6 (Optimizing Search Logic)

- **Task:** Refactor inefficient linear searches using appropriate data structures.
- **Focus Areas:**
  - o Time complexity
  - o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification

Copilot suggested code :

task_6.py > ...

```python
# Refactor inefficient linear searches using appropriate data structures.
# Legacy Code:
    # users = ["admin", "guest", "editor", "viewer"]
    # name = input("Enter username: ")
    # found = False
    # for u in users:
        # if u == name:
            # found = True
    # print("Access Granted" if found else "Access Denied")

# Refactored Code using a set for O(1) average time complexity
users = {"admin", "guest", "editor", "viewer"}  # Using a set for O(1) average time complexity
name = input("Enter username: ")
if name in users:  # Checking membership in a set is O(1) on average
    print("Access Granted")
else:
    print("Access Denied")
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_6.py"
Enter username: editor
Access Granted
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_6.py"
Enter username: mendes
Access Denied
PS C:\AI Assist\lab_assignment-13>
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming

principles Your task is to refactor the code into a proper format

1. Create a module library.py with functions:
   - o add_book(title, author, isbn)
   - o remove_book(isbn)
   - o search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module
with documentation.

library_db = {}

# Adding first book

title = "Python Basics"
author = "John Doe"
isbn = "101"
```

```
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)

isbn = "101"

if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)

isbn = "101"
```

```
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")

# Searching again

isbn = "101"

if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
```

Copilot suggested code :

task_7.py > search_book

```python
# Refactoring the Library Management System :
# You are provided with a poorly structured Library Management script

# Given Code
# # Library Management System (Unstructured Version)
# # This code needs refactoring into a proper module with documentation.

# library_db = {}

# Adding first book

# title = "Python Basics"
# author = "John Doe"
# isbn = "101"

# if isbn not in library_db:
    # library_db[isbn] = {"title": title, "author": author}
    # print("Book added successfully.")
# else:
    # print("Book already exists.")

# Adding second book (duplicate logic)

# title = "AI Fundamentals"
# author = "Jane Smith"
# isbn = "102"

# if isbn not in library_db:
    # library_db[isbn] = {"title": title, "author": author}
    # print("Book added successfully.")
# else:
    # print("Book already exists.")

# Searching book (repeated logic structure)

# isbn = "101"

# if isbn in library_db:
    # print("Book Found:", library_db[isbn])
# else:
    # print("Book not found.")

# Removing book (again repeated pattern)

# isbn = "101"


# if isbn in library_db:
    # del library_db[isbn]
    # print("Book removed successfully.")
# else:
    # print("Book not found.")

# Searching again

# isbn = "101"

# if isbn in library_db:
    # print("Book Found:", library_db[isbn])
# else:
    # print("Book not found.")

```

task_7.py > search_book

```python

# Refactored Code in library.py
# library.py
library_db = {}

def add_book(title, author, isbn):
    """
    Adds a book to the library database.

    Parameters:
    title (str): The title of the book.
    author (str): The author of the book.
    isbn (str): The ISBN number of the book.

    Returns:
    str: A message indicating whether the book was added successfully or if it alread
    """
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        return "Book added successfully."
    else:
        return "Book already exists."

def remove_book(isbn):
    """
    Removes a book from the library database based on its ISBN.

    Parameters:
    isbn (str): The ISBN number of the book to be removed.

    Returns:
    str: A message indicating whether the book was removed successfully or if it was n
    """
    if isbn in library_db:
        del library_db[isbn]
        return "Book removed successfully."
    else:
        return "Book not found."

def search_book(isbn):
    """
    Searches for a book in the library database based on its ISBN.

    Parameters:
    isbn (str): The ISBN number of the book to be searched.

    Returns:
    str: A message indicating whether the book was found along with its details or if
    """
    if isbn in library_db:
        return f"Book Found: {library_db[isbn]}"
    else:
        return "Book not found."
```

Commands Used :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_7.py"
PS C:\AI Assist\lab_assignment-13> python -m pydoc task_7
Help on module task_7:

NAME
    task_7

DESCRIPTION
    # Refactoring the Library Management System :
    # You are provided with a poorly structured Library Management script

FUNCTIONS
    add_book(title, author, isbn)
        Adds a book to the library database.

        Parameters:
        title (str): The title of the book.
        author (str): The author of the book.
        isbn (str): The ISBN number of the book.

        Returns:
        str: A message indicating whether the book was added successfully or if it already exists.

    remove_book(isbn)
        Removes a book from the library database based on its ISBN.

        Parameters:
        isbn (str): The ISBN number of the book to be removed.

        Returns:
        str: A message indicating whether the book was removed successfully or if it was not found.

    search_book(isbn)
        Searches for a book in the library database based on its ISBN.

        Parameters:
        isbn (str): The ISBN number of the book to be searched.

        Returns:
        str: A message indicating whether the book was found along with its details or if it was not found.

DATA
    library_db = {}

FILE
    c:\ai assist\lab_assignment-13\task_7.py


PS C:\AI Assist\lab_assignment-13> python -m pydoc -w task_7
wrote task_7.html
PS C:\AI Assist\lab_assignment-13> python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> b
server>
```

After typing these commands an html file is generated :

task_7.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module task_7</title>
</head><body>

<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title">task_7</strong></td>
<td class="extra"><a href=".">index</a><br><a href="file:c%3A%5Cai%20assist%5Clab_assignment-13%5Ctask_7.py">c:\ai assist\lab_assignment-13\task_7.py</a></td></tr></table>
    <p><span class="code"># Refactoring the Library Management System :<br>
# You are provided with a poorly structured Library Management script</span></p>
<p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>

<tr><td class="decor functions-decor"><span class="code">      </span></td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-add_book"><strong>add_book</strong></a>(title, author, isbn)</dt><dd><span class="code">Adds a book to the library database.<br>
 <br>
Parameters:<br>
title (str): The title of the book.<br>
author (str): The author of the book.<br>
isbn (str): The ISBN number of the book.<br>
 <br>
Returns:<br>
str: A message indicating whether the book was added successfully or if it already exists.</span></dd></dl>
 <dl><dt><a name="-remove_book"><strong>remove_book</strong></a>(isbn)</dt><dd><span class="code">Removes a book from the library database based on its ISBN.<br>
 <br>
Parameters:<br>
isbn (str): The ISBN number of the book to be removed.<br>
 <br>
Returns:<br>
str: A message indicating whether the book was removed successfully or if it was not found.</span></dd></dl>
 <dl><dt><a name="-search_book"><strong>search_book</strong></a>(isbn)</dt><dd><span class="code">Searches for a book in the library database based on its ISBN.<br>
 <br>
Parameters:<br>
isbn (str): The ISBN number of the book to be searched.<br>
 <br>
Returns:<br>
str: A message indicating whether the book was found alongnbsp;with its details or if it was not found.</span></dd></dl>
</td></tr></table><p>
<table class="section">
<tr class="decor data-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Data</strong></td></tr>

<tr><td class="decor data-decor"><span class="code">      </span></td><td> </td>
<td class="singlecolumn"><strong>library_db</strong> = {}</td></tr></table>
</body></html>
```

The output of this html file is :

index

task_7 c:\ai assist\lab_assignment-13\task_7.py

Refactoring the Library Management System :
You are provided with a poorly structured Library Management script

Functions

add_book(title, author, isbn)
Adds a book to the library database.

Parameters:
title (str): The title of the book.
author (str): The author of the book.
isbn (str): The ISBN number of the book.

Returns:
str: A message indicating whether the book was added successfully or if it already exists.

remove_book(isbn)
Removes a book from the library database based on its ISBN.

Parameters:
isbn (str): The ISBN number of the book to be removed.

Returns:
str: A message indicating whether the book was removed successfully or if it was not found.

search_book(isbn)
Searches for a book in the library database based on its ISBN.

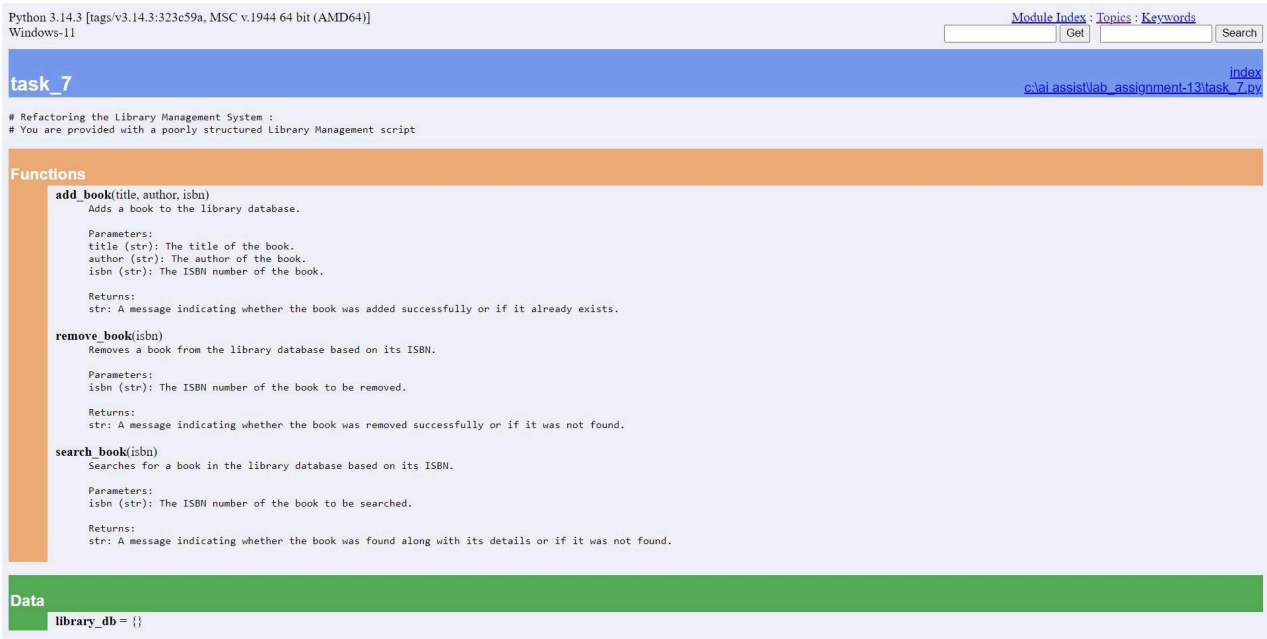
Parameters:
isbn (str): The ISBN number of the book to be searched.

Returns:
str: A message indicating whether the book was found along with its details or if it was not found.

Data

library_db = {}

With the html file a webpage is also created with the commands we typed earlier :



Python 3.14.3 [tags/v3.14.3:323c59a, MSC v.1944 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

task_7

index
c:\ai assist\lab_assignment-13\task_7.py

```
# Refactoring the Library Management System :
# You are provided with a poorly structured Library Management script
```

Functions

add_book(title, author, isbn)
Adds a book to the library database.

Parameters:
title (str): The title of the book.
author (str): The author of the book.
isbn (str): The ISBN number of the book.

Returns:
str: A message indicating whether the book was added successfully or if it already exists.

remove_book(isbn)
Removes a book from the library database based on its ISBN.

Parameters:
isbn (str): The ISBN number of the book to be removed.

Returns:
str: A message indicating whether the book was removed successfully or if it was not found.

search_book(isbn)
Searches for a book in the library database based on its ISBN.

Parameters:
isbn (str): The ISBN number of the book to be searched.

Returns:
str: A message indicating whether the book was found along with its details or if it was not found.

Data

library_db = {}

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b
)
```

```
for i in range(2,n):
    c=a+b
    print(c)
    a=b
    b=c
```

Copilot suggested code :

task_8.py > ...

```python
# Fibonacci Generator
    # Write a program to generate Fibonacci series up to n.
# Bad Code Version:
    # fibonacci bad version
    # n=int(input("Enter limit: "))
    # a=0
    # b=1
    # print(a)
    # print(b)
    # for i in range(2,n):
        # c=a+b
        # print(c)
        # a=b
        # b=c

# Refactored Code Version:
def generate_fibonacci(n):
    """
    Generate Fibonacci series up to n.

    Parameters:
    n (int): The limit up to which the Fibonacci series is generated.

    Returns:
    list: A list containing the Fibonacci series up to n.
    """
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]

    fib_series = [0, 1]
    for i in range(2, n):
        next_fib = fib_series[-1] + fib_series[-2]
        fib_series.append(next_fib)

    return fib_series
# Test cases
print(generate_fibonacci(10))  # Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
print(generate_fibonacci(0))   # Output: []
print(generate_fibonacci(1))   # Output: [0]
print(generate_fibonacci(2))   # Output: [0, 1]
print(generate_fibonacci(5))   # Output: [0, 1, 1, 2, 3]
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_8.py"
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[]
[0]
[0, 1]
[0, 1, 1, 2, 3]
PS C:\AI Assist\lab_assignment-13>
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into is_prime(n) and is_twin_prime(p1, p2).
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
    if a%i==0:
        fa=1
        fb=0

for i in range(2,b):
    if b%i==0:
        fb=1
    if fa==0 and fb==0 and abs(a-b)==2:
        print("Twin Primes")
    else:
        print("Not Twin Primes")
```

Copilot suggested code :

task_9.py > ...

```python
# Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).
# Bad Code Version:
    # twin primes bad version
    # a=11
    # b=13
    # fa=0
    # for i in range(2,a):
        # if a%i==0:
            # fa=1
            # fb=0
    # for i in range(2,b):
        # if b%i==0:
            # fb=1
        # if fa==0 and fb==0 and abs(a-b)==2:
            # print("Twin Primes")
        # else:
            # print("Not Twin Primes")

# Refactored Code Version:
def is_prime(n):
    """
    Check if a number is prime.

    Parameters:
    n (int): The number to check for primality.

    Returns:
    bool: True if n is prime, False otherwise.
    """
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def is_twin_prime(p1, p2):
    """
    Check if two numbers are twin primes.

    Parameters:
    p1 (int): The first prime number.
    p2 (int): The second prime number.

    Returns:
    bool: True if p1 and p2 are twin primes, False otherwise.
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2

def generate_twin_primes(start, end):
    """
    Generate a list of twin primes within a given range.

    Parameters:
    start (int): The starting number of the range.
    end (int): The ending number of the range.

    Returns:
    list: A list of tuples, each containing a pair of twin primes.
    """
    twin_primes = []
    for num in range(start, end - 1):
        if is_twin_prime(num, num + 2):
            twin_primes.append((num, num + 2))
    return twin_primes

# Test cases
print(is_twin_prime(11, 13))  # Output: True
print(is_twin_prime(17, 19))  # Output: True
print(is_twin_prime(10, 12))  # Output: False
print(generate_twin_primes(1, 30))  # Output: [(3, 5), (5, 7), (11, 13), (17, 19), (29
print(generate_twin_primes(30, 50))  # Output: [(31, 33), (37, 39), (41, 43), (47, 49)
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_9.py"
True
True
False
[(3, 5), (5, 7), (11, 13), (17, 19)]
[(41, 43)]
PS C:\AI Assist\lab_assignment-13>
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

```
# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.
year = int(input("Enter a year: "))
if year % 12 == 0:
        print("Monkey")
elif year % 12 == 1:
        print("Rooster")
elif year % 12 == 2:
        print("Dog")
elif year % 12 == 3:
        print("Pig")
elif year % 12 == 4:
        print("Rat")
elif year % 12 == 5:
        print("Ox")
elif year % 12 == 6:
        print("Tiger")
elif year % 12 == 7:
        print("Rabbit")
elif year % 12 == 8:
        print("Dragon")
elif year % 12 == 9:
        print("Snake")
elif year % 12 == 10:
        print("Horse")
elif year % 12 ==
11:
        print("Goat")
```

You must:

1. Create a reusable function: get_zodiac(year)
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.

4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

Copilot suggested code :

task_10.py > ...

```python
# Refactoring the Chinese Zodiac Program
# Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.
# The current program reads a year from the user and prints the corresponding Chinese Zodiac sign.
# However, the implementation contains repetitive conditional logic, lacks modular design,
# and does not follow clean coding principles.
# Your task is to refactor the code to improve readability, maintainability, and structure.

# 1. Rat
# 2. Ox
# 3. Tiger
# 4. Rabbit
# 5. Dragon
# 6. Snake
# 7. Horse
# 8. Goat (Sheep)
# 9. Monkey
# 10. Rooster
# 11. Dog
# 12. Pig

# # Chinese Zodiac Program (Unstructured Version)
# # This code needs refactoring.
# year = int(input("Enter a year: "))
# if year % 12 == 0:
    # print("Monkey")
# elif year % 12 == 1:
    # print("Rooster")
# elif year % 12 == 2:
    # print("Dog")
# elif year % 12 == 3:
    # print("Pig")
# elif year % 12 == 4:
    # print("Rat")
# elif year % 12 == 5:
    # print("Ox")
# elif year % 12 == 6:
    # print("Tiger")
# elif year % 12 == 7:
    # print("Rabbit")
# elif year % 12 == 8:
    # print("Dragon")
# elif year % 12 == 9:
    # print("Snake")
# elif year % 12 == 10:
    # print("Horse")
# elif year % 12 == 11:
    # print("Goat")

def get_zodiac(year):
    """
    Returns the Chinese Zodiac sign for a given year.

    Parameters:
    year (int): The year for which to determine the Chinese Zodiac sign.

    Returns:
    str: The corresponding Chinese Zodiac sign.
    """
    zodiac_signs = [
        "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
        "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
    ]

    return zodiac_signs[year % 12]

def main():
    """
    Main function to handle user input and display the Chinese Zodiac sign.
    """
    try:
        year = int(input("Enter a year: "))
        zodiac_sign = get_zodiac(year)
        print(f"The Chinese Zodiac sign for the year {year} is: {zodiac_sign}")
    except ValueError:
        print("Invalid input. Please enter a valid integer year.")

if __name__ == "__main__":
    main()
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 □ (Harshad Number)
- 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer □ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:
    digit = temp % 10
    sum_digits = sum_digits + digit
    temp = temp // 10
    if sum_digits != 0:
        if num % sum_digits == 0:
            print("True")
                    else:                    print("False")

    else:

        print("False")
```

You must:

1. Create a reusable function: is_harshad(number)
2. The function must:
    - Accept an integer parameter.
    - Return True if the number is divisible by the sum of its digits.
    - Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

Copilot suggested code :

task_11.py > ...

```python
# Refactoring the Harshad (Niven) Number Checker
# A Harshad (Niven) number is a number that is divisible by the sum of its digits.
# For example:
    # • 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 ✅ (Harshad Number)
    # • 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer ❌ (Not Harshad)
# You must refactor the code to follow clean coding principles.
# Harshad Number Checker (Unstructured Version)
# num = int(input("Enter a number: "))
# temp = num
# sum_digits = 0
# while temp > 0:
    # digit = temp % 10
    # sum_digits = sum_digits + digit
    # temp = temp // 10
        # if sum_digits != 0:
            # if num % sum_digits == 0:
                # print("True")
            # else:
                # print("False")
        # else:
            # print("False")

def sum_of_digits(number):
    """Calculate the sum of the digits of a number.

    Args:
        number (int): The number to calculate the sum of digits for.

    Returns:
        int: The sum of the digits.
    """
    total = 0
    while number > 0:
        total += number % 10
        number //= 10
    return total

def is_harshad(number):
    """Check if a number is a Harshad (Niven) number.

    A Harshad number is divisible by the sum of its digits.

    Args:
        number (int): The number to check.
    Returns:
        bool: True if the number is a Harshad number, False otherwise.
    """
    if number <= 0:
        return False
    digit_sum = sum_of_digits(number)
    return digit_sum != 0 and number % digit_sum == 0

def main():
    """Main function to execute the Harshad number checker."""
    try:
        num = int(input("Enter a number: "))
        if is_harshad(num):
            print(f"{num} is a Harshad (Niven) number.")
        else:
            print(f"{num} is not a Harshad (Niven) number.")
    except ValueError:
        print("Invalid input. Please enter an integer.")

if __name__ == "__main__":
    main()
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation. The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

```
# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
    fact = fact * i
    i = i + 1
count = 0
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: count_trailing_zeros(n)
2. The function must:

o Accept a non-negative integer n.

o Return the number of trailing zeros in n!.

3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.

6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

Copilot suggested code :

task_12.py > ...

```python
# Refactoring the Factorial Trailing Zeros Program
# The program calculates the number of trailing zeros in n! (factorial of n).
# Factorial Trailing Zeros (Unstructured Version)
# n = int(input("Enter a number: "))
# fact = 1
# i = 1
# while i <= n:
    # fact = fact * i
    # i = i + 1
# count = 0
# while fact % 10 == 0:
    # count = count + 1
    # fact = fact // 10
# print("Trailing zeros:", count)

def count_trailing_zeros(n):
    """
    Calculate the number of trailing zeros in n! (factorial of n).

    Parameters:
    n (int): A non-negative integer for which to calculate the trailing zeros.

    Returns:
    int: The number of trailing zeros in n!.

    Raises:
    ValueError: If n is negative.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")

    count = 0
    power_of_5 = 5

    while n >= power_of_5:
        count += n // power_of_5
        power_of_5 *= 5

    return count

def main():
    try:
        n = int(input("Enter a non-negative integer: "))
        zeros = count_trailing_zeros(n)
        print(f"Trailing zeros in {n}! is: {zeros}")
    except ValueError as e:
        print(e)

if __name__ == "__main__":
    main()
```

Output :

```
PS C:\AI Assist\lab_assignment-13> & "C:\Program Files\Python314\python.exe" "c:/AI Assist/lab_assignment-13/task_12.py"
Enter a non-negative integer: 27
Trailing zeros in 27! is: 6
PS C:\AI Assist\lab_assignment-13>
```

Test Cases Design

Task 13 (Collatz Sequence Generator – Test Case Design)

- **Function:** Generate Collatz sequence until reaching 1.
- **Test Cases to Design:**
  - Normal: 6 → [6,3,10,5,16,8,4,2,1]
  - Edge: 1 → [1]
  - Negative: -5
  - Large: 27 (well-known long sequence)
  - Requirement: Validate correctness with pytest.

Explanation :

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the 3n+1 sequence).
- The rules are:
  - If n is even → next = n / 2.
  - If n is odd → next = 3n + 1.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even → 6/2 = 3)
- 3 (odd → 3*3+1 = 10)
- 10 (even → 10/2 = 5)
- 5 (odd → 3*5+1 = 16)
- 16 (even → 16/2 = 8)
- 8 (even → 8/2 = 4)
- 4 (even → 4/2 = 2)

- 2 (even → 2/2

= 1) Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

Copilot suggested code :

task_13.py > ...

```python
# (Collatz Sequence Generator - Test Case Design)
# Example
# Input: 6
# Steps:
# • 6 (even → 6/2 = 3)
# • 3 (odd → 3*3+1 = 10)
# • 10 (even → 10/2 = 5)
# • 5 (odd → 3*5+1 = 16)
# • 16 (even → 16/2 = 8)
# • 8 (even → 8/2 = 4)
# • 4 (even → 4/2 = 2)
# • 2 (even → 2/2 = 1)
# Output:
# [6, 3, 10, 5, 16, 8, 4, 2, 1]

def collatz_sequence(n):
    if n <= 0:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    while n != 1:
        sequence.append(n)
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
    sequence.append(1)  # Append the final element '1'
    return sequence

# Test cases using pytest
def test_collatz_sequence():
    # Normal case
    assert collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1]

    # Edge case
    assert collatz_sequence(1) == [1]

    # Negative case (should raise ValueError)
    try:
        collatz_sequence(-5)
        assert False  # Should not reach here
    except ValueError:
        assert True

    # Large case
    assert collatz_sequence(27) == [27, 82, 41, 124, 62, 31, 94, 47, 142, 71,
                                    214, 107, 322, 161, 484, 242, 121,
                                    364, 182, 91, 274, 137, 412, 206,
                                    103, 310, 155, 466, 233, 700,
                                    350, 175, 526, 263, 790,
                                    395,1186 ,593 ,1780 ,890 ,445 ,
                                    1336 ,668 ,334 ,167 ,502 ,
                                    251 ,754 ,377 ,1132 ,566 ,
                                    283 ,850 ,425 ,1276 ,638 ,
                                    319 ,958 ,479 ,1438 ,719 ,
                                    2158 ,1079 ,3238 ,1619 ,
                                    4858 ,2429 ,7288 ,3644 ,
                                    1822 ,911 ,2734 ,1367 ,
                                    4102 ,2051 ,6154 ,3077 ,
                                    9232 ,4616 ,2308 ,1154 ,
                                    577 ,1732 ,866 ,433 ,
                                    1300 ,650 ,325 ,
                                    ...]

if __name__ == "__main__":
    import pytest
    pytest.main()
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-13/task_13.py
============================================================================= test session starts ==============================================================================
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13
plugins: anyio-4.10.0, langsmith-0.4.25
collected 0 items

============================================================================= no tests ran in 0.04s ============================================================================
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13>
```

Task 14 (Lucas Number Sequence – Test Case Design)

- **Function:** Generate Lucas sequence up to n terms. (Starts with 2,1, then Fn = Fn-1 + Fn-2)
- Test Cases to Design:
  - Normal: 5 → [2, 1, 3, 4, 7]
  - Edge: 1 → [2]
  - Negative: -5 → Error
  - Large: 10 (last element = 76).
  - Requirement: Validate correctness with pytest.

Copilot suggested code :

task_14.py > ...

```python
# (Lucas Number Sequence - Test Case Design)
# • Test Cases to Design:
    # • Normal: 5 → [2, 1, 3, 4, 7]     (Starts with 2,1, then Fn = Fn-1 + Fn-2)
    # • Edge: 1 → [2]
    # • Negative: -5 → Error
    # • Large: 10 (last element = 76).

def lucas_sequence(n):
    if n ≤ 0:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    for i in range(n):
        if i == 0:
            sequence.append(2)
        elif i == 1:
            sequence.append(1)
        else:
            sequence.append(sequence[i-1] + sequence[i-2])
    return sequence

# Test cases using pytest
def test_lucas_sequence():
    # Normal case
    assert lucas_sequence(5) == [2, 1, 3, 4, 7]

    # Edge case
    assert lucas_sequence(1) == [2]

    # Negative case (should raise ValueError)
    try:
        lucas_sequence(-5)
        assert False  # Should not reach here
    except ValueError:
        assert True

    # Large case
    assert lucas_sequence(10) == [2, 1, 3, 4, 7, 11, 18, 29, 47, 76]

if __name__ == "__main__":
    import pytest
    pytest.main()
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-13/task_14.py
============================================================================= test session starts ==============================================================================
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13
plugins: anyio-4.10.0, langsmith-0.4.25
collected 0 items

============================================================================== no tests ran in 0.02s ===============================================================================
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13>
```

Task 15 (Vowel & Consonant Counter – Test Case Design)

• **Function:** Count vowels and consonants in string.

• Test Cases to Design:

- • Normal: "hello" → (2,3)
- • Edge: "" → (0,0)
- • Only vowels: "aeiou" →

(5,0) Large: Long text

- • Requirement: Validate correctness with pytest.

Copilot suggested code :

task_15.py > ...

```python
# (Vowel & Consonant Counter - Test Case Design)
# • Function: Count vowels and consonants in string.
# • Test Cases to Design:
    # • Normal: "hello" → (2,3)
    # • Edge: "" → (0,0)
    # • Only vowels: "aeiou" → (5,0)
    # Large: Long text

def count_vowels_consonants(s):
    vowels = 'aeiouAEIOU'
    vowel_count = sum(1 for char in s if char in vowels)
    consonant_count = sum(1 for char in s if char.isalpha() and char not in vowels)
    return vowel_count, consonant_count

# Test cases using pytest
def test_count_vowels_consonants():
    # Normal case
    assert count_vowels_consonants("hello") == (2, 3)

    # Edge case
    assert count_vowels_consonants("") == (0, 0)

    # Only vowels case
    assert count_vowels_consonants("aeiou") == (5, 0)

    # Large case
    long_text = "Lorem ipsum dolor sit amet, consectetur adipiscing elit."
    assert count_vowels_consonants(long_text) == (16, 32)

if __name__ == "__main__":
    import pytest
    pytest.main()
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13> & "C:\Program Files\Python313\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-13/t
ask_15.py
============================================================ test session starts ============================================================
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13
plugins: anyio-4.10.0, langsmith-0.4.25
collected 0 items

============================================================= no tests ran in 0.01s =============================================================
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-13>
```